

AWS Python Deployment

Session 1 — Server-Based Deployment

AmaliTech · Python Chapter

2025

Contents

Before You Start

Demo 1 — Single EC2 Instance Running Flask

Demo 2 — Load Balancer with Auto Scaling Group

Demo 3 — Automated Deployment with AWS CodeDeploy

Cleanup — Delete everything we created

Section A — CI/CD with GitHub Actions (optional, ~20 minutes)

Before You Start

What you need on your laptop

- An AWS account where you can create resources (the AWS Free Tier is enough)
- A web browser to use the AWS Management Console
- A terminal (this guide assumes **Ubuntu Linux** — every shell command works as-is)
- Python 3.12 installed for local testing
- A free GitHub account

What we will deploy

A small Python Flask application with three endpoints:

- `GET /` — returns a greeting
- `POST /echo` — echoes back JSON we send
- `GET /health` — returns `ok`, used by the load balancer

We use the **AWS Management Console** for every step in these demos. The Console is the visual interface at <https://console.aws.amazon.com>. We use it because:

- It is the easiest way to [see](#) what each AWS service looks like
- Every click in the Console maps to one API call — once you know the Console, the AWS CLI and Terraform make sense
- Mistakes are obvious and easy to undo

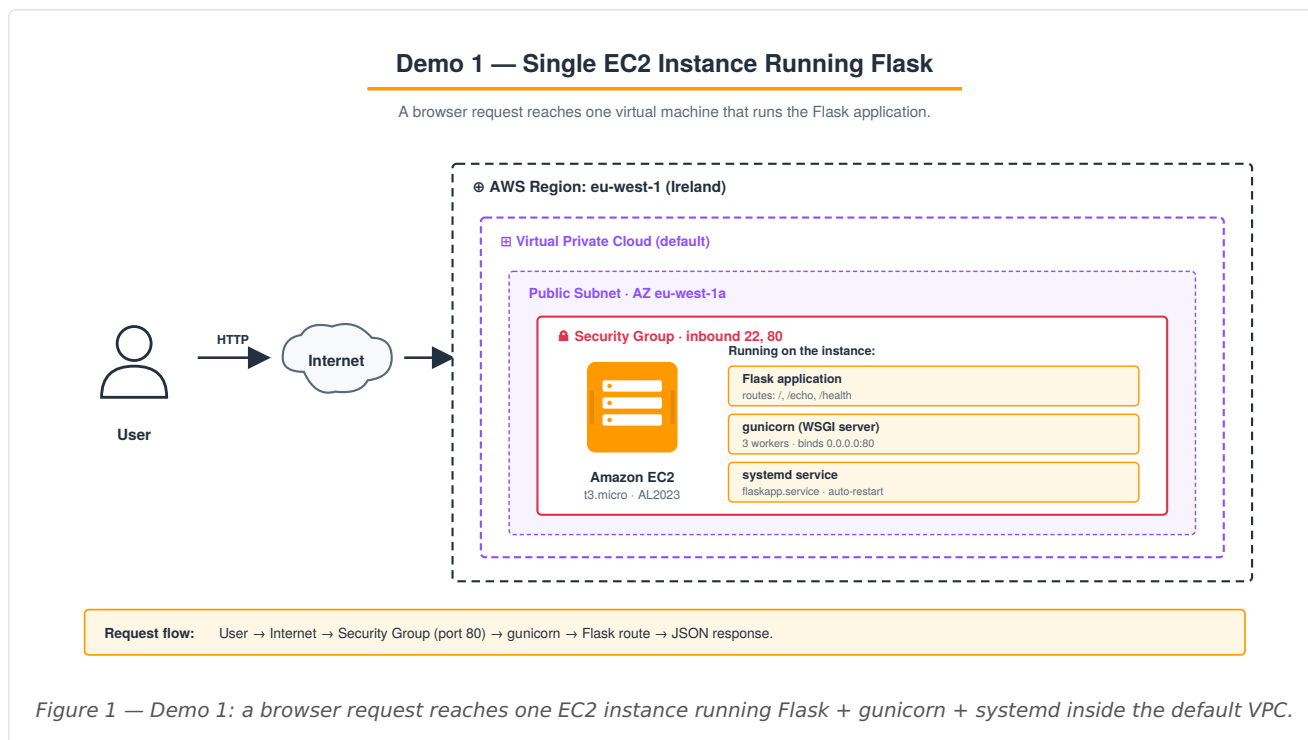
The Region we use

All three demos use `eu-west-1` (Ireland). It is the closest AWS Region to Ghana with full service availability. Set the Region in the top-right corner of the Console before doing anything.

Tip: put every resource you create today under a project tag so cleanup at the end is easy. We use `Project = python-chapter-demo`. The Console has a "Tags" section on every "Create..." form.

Demo 1 — Single EC2 Instance Running Flask

Goal: run our Flask app on one Linux virtual machine in AWS, reachable from any browser.



The project structure

We build this on our laptop first, test it locally, then push to GitHub and deploy.

```
flask-app/  
├─ app.py           # The Flask application  
├─ requirements.txt # Python dependencies  
└─ README.md       # Notes for your team
```

Step 1.1 — Create the local project and the Flask app

Open a terminal on your laptop and run:

```
# Create the project folder  
mkdir flask-app  
cd flask-app
```

Now create the application file. Open your editor (VS Code, nano, vim, etc.) and create `app.py` with this content:

File: `app.py`

```

from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route("/")
def index():
    return jsonify({"message": "Hello from Flask on AWS!"})

@app.route("/echo", methods=["POST"])
def echo():
    # Echo back whatever JSON was sent
    data = request.get_json(silent=True) or {}
    return jsonify({"you_sent": data})

@app.route("/health")
def health():
    # Used by the load balancer in Demo 2
    return "ok", 200

if __name__ == "__main__":
    # Bind to 0.0.0.0 so it is reachable from outside the machine
    app.run(host="0.0.0.0", port=8000)

```

Create the dependencies file:

File: requirements.txt

```

flask==3.0.3
gunicorn==23.0.0

```

Why gunicorn? Flask's built-in server (`app.run`) is only for development — it is single-threaded and warns you not to use it in production. **gunicorn** is a production-grade WSGI server that runs multiple Python worker processes so the app can handle many requests at once.

Step 1.2 — Test the app locally before deploying

Always run the app on your laptop first. If it does not work locally, it will not work on AWS.

```

# If Python or venv is missing, install them first
sudo apt update
sudo apt install -y python3 python3-venv python3-pip

# Create a Python virtual environment
python3 -m venv venv

# Activate it
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Run the app
python app.py

```

You should see:

- * Running on all addresses (0.0.0.0)
- * Running on http://127.0.0.1:8000

Open a **second terminal** (keep the app running in the first) and test:

```
# Test the root endpoint
curl http://localhost:8000/

# Test the echo endpoint
curl -X POST http://localhost:8000/echo \
  -H "Content-Type: application/json" \
  -d '{"name": "Ghana"}'

# Test the health endpoint
curl http://localhost:8000/health
```

You should see JSON responses for `/` and `/echo`, and `ok` for `/health`.

Stop the app with `Ctrl+C` in the first terminal. Local test passed — we can deploy.

Step 1.3 — Push the code to GitHub

CodeDeploy in Demo 3 will pull from a release zip, but for Demo 1 we **clone the code directly from GitHub onto the EC2 instance**. So push it now.

```
# Add a .gitignore so we do not commit the venv
echo "venv/" > .gitignore
echo "__pycache__/" >> .gitignore

# Initialise git
git init
git add .
git commit -m "Initial Flask app"

# Create a new public repo on GitHub called "flask-app"
# (use https://github.com/new in your browser)
# Then point your local repo to it:
git branch -M main
git remote add origin https://github.com/<your-username>/flask-app.git
git push -u origin main
```

Replace `<your-username>` with your GitHub username. The repo must be **public** so the EC2 instance can clone it without authentication.

Step 1.4 — Create an SSH key pair in the AWS Console

We need a key pair to SSH into the EC2 instance later.

In the AWS Console:

1. Make sure the Region in the top-right is **eu-west-1**
2. Search for **EC2** in the top search bar and open it
3. In the left sidebar under **Network & Security**, click **Key Pairs**
4. Click **Create key pair** (top right)
5. Name: `flask-demo-key`
6. Key pair type: **RSA**
7. Private key file format: **.pem**
8. Click **Create key pair**

Your browser will download `flask-demo-key.pem`. Move it to a safe location and lock down its permissions:

```
mkdir -p ~/.ssh
mv ~/Downloads/flask-demo-key.pem ~/.ssh/
chmod 400 ~/.ssh/flask-demo-key.pem
```

Why `chmod 400`? SSH refuses to use a key file that is readable by anyone else on the machine. `400` means only you can read it.

Step 1.5 — Create a Security Group

A **Security Group** is a virtual firewall around the EC2 instance. It controls which ports are reachable and from where.

In the AWS Console:

1. Still in **EC2**, in the left sidebar under **Network & Security**, click **Security Groups**
2. Click **Create security group** (top right)
3. Name: `flask-demo-sg`
4. Description: `Security group for the Flask demo`
5. VPC: leave the default

Under **Inbound rules**, click **Add rule** twice and configure:

Type	Protocol	Port range	Source	Description
SSH	TCP	22	My IP	SSH from my laptop
HTTP	TCP	80	Anywhere-IPv4 (0.0.0.0/0)	HTTP from anywhere

Under **Outbound rules** leave the default (all traffic allowed out).

Scroll down, in **Tags**, click **Add new tag**: Key `Project`, Value `python-chapter-demo`.

Click **Create security group**.

Why two rules? Port 22 is SSH — we only allow it from our IP so randoms on the internet cannot try to log in. Port 80 is HTTP — anyone in the world should be able to view our app.

Step 1.6 — Launch an EC2 instance

Now we create the virtual machine itself.

In the AWS Console:

1. Still in **EC2**, in the left sidebar click **Instances**
2. Click **Launch instances** (orange button, top right)
3. **Name and tags:** `flask-demo-instance`. Click **Add additional tags** -> Key `Project`, Value `python-chapter-demo`.
4. **Application and OS Images (AMI):** in the **Quick Start** tab select **Ubuntu**, then in the AMI dropdown choose **Ubuntu Server 22.04 LTS (HVM), SSD Volume Type** — make sure it says `64-bit (x86)`
5. **Instance type:** `t3.micro` (Free Tier eligible)
6. **Key pair (login):** select `flask-demo-key`
7. **Network settings:** click **Edit** on the right.
 - Auto-assign public IP: **Enable**
 - Firewall: select **Select existing security group**
 - Common security groups: select `flask-demo-sg`
8. **Configure storage:** leave at 8 GB gp3
9. **Advanced details:** scroll down to **User data** and paste the script below

Then click **Launch instance** (bottom right).

This is the **user data** script — paste it into the "User data" text area in step 9:

```
#!/bin/bash
# Runs once on first boot, as root

set -e

# Update the system and install Python + git
apt update -y
apt install -y python3 python3-venv python3-pip git

# Clone the application repository
cd /home/ubuntu
git clone https://github.com/<your-username>/flask-app.git
chown -R ubuntu:ubuntu flask-app

# Install dependencies into a venv
cd flask-app
python3 -m venv venv
./venv/bin/pip install -r requirements.txt

# Create the systemd unit file
cat > /etc/systemd/system/flaskapp.service <<EOF
[Unit]
Description=Flask App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/flask-app
ExecStart=/home/ubuntu/flask-app/venv/bin/gunicorn \\\
    --workers 3 --bind 0.0.0.0:80 app:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF

# Start the service and enable it on boot
systemctl daemon-reload
systemctl enable flaskapp
systemctl start flaskapp
```

Replace `<your-username>` with your actual GitHub username before pasting.

What is User Data? It is a shell script that AWS runs the first time the instance boots. We use it to install Python, clone the code, install dependencies, and start the app — all without touching the machine manually.

Why systemd? It makes our app a real Linux service. If the app crashes, systemd restarts it. If the machine reboots, the app comes back up automatically. This is the production pattern on Linux.

Step 1.7 — Wait, then test the app

Back on the **Instances** page, click your instance and wait until the **Status check** column shows `2/2 checks passed` (about 2-3 minutes). Then:

1. Copy the **Public IPv4 address** of the instance (shown in the Details panel)
2. Open `http://<public-ip>/` in your browser

You should see:

```
{"message": "Hello from Flask on AWS!"}
```

If you see the JSON message — your Flask app is live on AWS.

Test the other endpoints from your laptop:

```
PUBLIC_IP=<your-instance-public-ip>

curl http://$PUBLIC_IP/
curl http://$PUBLIC_IP/health
curl -X POST http://$PUBLIC_IP/echo \
  -H "Content-Type: application/json" \
  -d '{"name": "AmaliTech"}'
```

If the page does not load: wait another minute. The user-data script takes 60–90 seconds to install Python, clone the repo, and start gunicorn. If it still does not load after 3 minutes, SSH in and check (see next step).

Step 1.8 — SSH in to inspect (optional)

```
ssh -i ~/.ssh/flask-demo-key.pem ubuntu@<public-ip>

# Once connected:
sudo systemctl status flaskapp    # Is the service running?
sudo journalctl -u flaskapp -n 50 # Last 50 log lines
cat /var/log/cloud-init-output.log # Output of the user data script
```

Type `exit` to leave the SSH session.

What we built in Demo 1

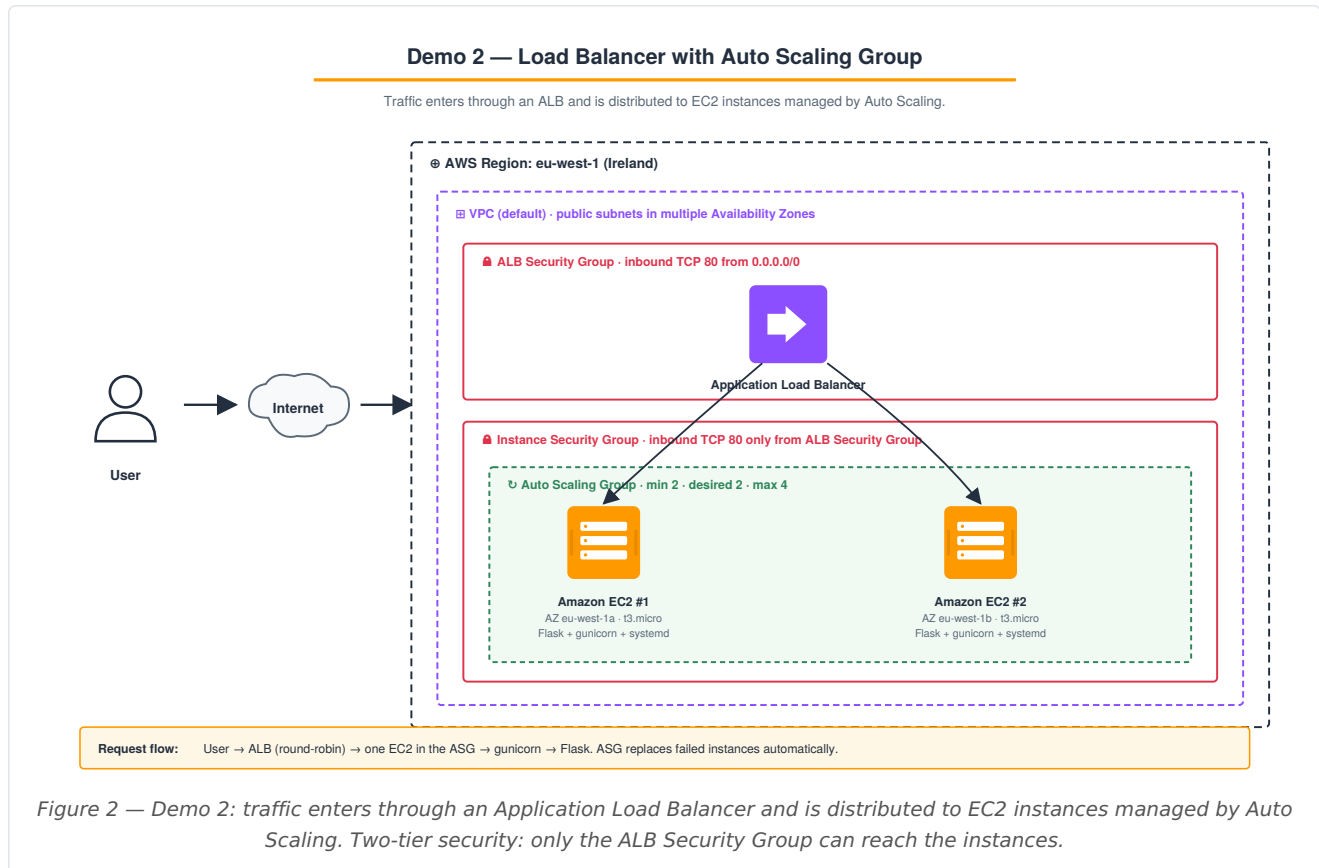
A Flask application running on one EC2 instance in `eu-west-1`. It is reachable from anywhere on the internet on port 80. It auto-restarts if it crashes, and starts automatically if the machine reboots.

The architecture is simple — but it has one EC2 instance. If that instance fails, the app goes down. **Demo 2 fixes that** by adding a load balancer and multiple instances.

Do not terminate this instance yet. Demo 2 builds on top of it.

Demo 2 — Load Balancer with Auto Scaling Group

Goal: distribute traffic across multiple EC2 instances, replace failed instances automatically, and handle increased load by adding more instances.



What changes from Demo 1

The Flask application code does not change. What changes is the *infrastructure*:

- We add an **Application Load Balancer (ALB)** in front of the instances
- We replace the single instance with an **Auto Scaling Group (ASG)** that runs 2 to 4 instances
- We add a **second Security Group** so only the ALB can reach the instances (not the open internet)

The code is reused from Demo 1's GitHub repo.

Step 2.1 — Create the ALB Security Group

We need a separate Security Group for the ALB, distinct from the one for the instances.

In the AWS Console:

1. **EC2 -> Security Groups -> Create security group**

2. Name: `flask-alb-sg`

3. Description: `Security group for the ALB`

4. VPC: default

Inbound rule: Type `HTTP`, Source `Anywhere-IPv4`

Tags: `Project = python-chapter-demo`

Create security group.

Step 2.2 — Update the instance Security Group

We change `flask-demo-sg` so that only the ALB can reach the instances on port 80.

In the AWS Console:

1. **Security Groups -> select `flask-demo-sg` -> Inbound rules tab -> Edit inbound rules**

2. Find the HTTP rule (port 80, source `0.0.0.0/0`) and click **Delete** on its row

3. Click **Add rule:**

- Type: `HTTP`

- Source: **Custom**, then in the search box select `flask-alb-sg`

- Description: `From ALB only`

4. Click **Save rules**

The SSH rule stays — we can still SSH in from our laptop.

Why this matters: the instances are no longer reachable from the internet directly. The only way in for HTTP traffic is through the ALB. This is the **two-tier security pattern** — public-facing edge + protected internal tier.

Step 2.3 — Create a Target Group

A **Target Group** is the bridge between the ALB and the instances. The ALB sends traffic to the Target Group, and the Target Group keeps track of which instances are healthy.

In the AWS Console:

1. **EC2** -> in the left sidebar under **Load Balancing**, click **Target Groups**
2. **Create target group**
3. Choose target type: **Instances**
4. Name: `flask-demo-tg`
5. Protocol: `HTTP`, Port: `80`
6. VPC: default
7. Protocol version: `HTTP1`
8. **Health checks:**
 - Protocol: `HTTP`
 - Path: `/health`
9. Advanced health check settings (expand):
 - Healthy threshold: `2`
 - Unhealthy threshold: `2`
 - Interval: `15` seconds
 - Timeout: `5` seconds
10. Tags: `Project = python-chapter-demo`
11. **Next** -> on the "Register targets" page, do not add any targets yet -> **Create target group**

What is a health check? Every 15 seconds, the ALB asks each instance: "is `/health` returning 200 OK?" If two consecutive checks fail, the instance is marked unhealthy and the ALB stops sending it traffic until it recovers.

Step 2.4 — Create the Application Load Balancer

In the AWS Console:

1. **EC2** -> in the left sidebar under **Load Balancing**, click **Load Balancers**
2. **Create load balancer** -> select **Application Load Balancer** -> **Create**
3. Name: `flask-demo-alb`
4. Scheme: **Internet-facing**
5. IP address type: **IPv4**
6. **Network mapping:**
 - VPC: default
 - Mappings: tick **all** Availability Zones (usually 3 in eu-west-1)
7. **Security groups:** remove `default`, add `flask-alb-sg`
8. **Listeners and routing:**
 - Protocol: `HTTP`, Port: `80`
 - Default action: **Forward to** `flask-demo-tg`
9. Tags: `Project = python-chapter-demo`
10. **Create load balancer**

Provisioning takes 2-4 minutes. While we wait, build the Launch Template.

Step 2.5 — Create a Launch Template

A **Launch Template** is a saved blueprint for new EC2 instances. The Auto Scaling Group uses it to know *how* to create instances when scaling up.

In the AWS Console:

1. **EC2** -> in the left sidebar under **Instances**, click **Launch Templates**
2. **Create launch template**
3. Name: `flask-demo-lt`
4. Template version description: `Flask on Ubuntu 22.04 LTS`
5. **Application and OS Images:** in the **Quick Start** tab select **Ubuntu**, then choose **Ubuntu Server 22.04 LTS (HVM), SSD Volume Type** (64-bit x86)
6. **Instance type:** `t3.micro`
7. **Key pair:** `flask-demo-key`
8. **Network settings:** leave subnet at "Don't include in launch template" (the ASG will choose)
9. **Security groups:** `flask-demo-sg`
10. **Resource tags:** click **Add tag:** Key `Project`, Value `python-chapter-demo`. **Resource types:** select `Instances` and `Volumes`.
11. Expand **Advanced details** -> scroll to **User data** at the bottom -> paste the **same user-data script from Demo 1 Step 1.6** (remember to change `<your-username>` to your GitHub username)
12. **Create launch template**

Step 2.6 — Create the Auto Scaling Group

In the AWS Console:

1. **EC2** -> in the left sidebar under **Auto Scaling**, click **Auto Scaling Groups**
2. **Create Auto Scaling group**
3. Name: `flask-demo-asg`
4. Launch template: `flask-demo-lt` -> **Next**
5. **Network:**
 - VPC: default
 - Availability Zones and subnets: tick **all** default subnets -> **Next**
6. **Load balancing:** select **Attach to an existing load balancer**
 - **Choose from your load balancer target groups:** `flask-demo-tg`
 - Health checks: tick **Turn on Elastic Load Balancing health checks**
 - Health check grace period: `120` seconds
 - **Next**
7. **Configure group size:**
 - Desired capacity: `2`
 - Minimum: `2`
 - Maximum: `4`
 - **Next**
8. Notifications: skip -> **Next**
9. Tags: add `Project = python-chapter-demo`, tick **Tag new instances** -> **Next**
10. Review -> **Create Auto Scaling group**

What just happened: the ASG immediately starts launching 2 instances using the Launch Template. Each instance runs the user-data script (installs Python, clones the repo, starts the Flask service). After about 2-3 minutes, both instances will be running and healthy.

Step 2.7 — Test the load balancer

Wait 3-4 minutes for the instances to launch and pass health checks. Then:

1. **EC2** -> **Load Balancers** -> click `flask-demo-alb`
2. In the Details panel, copy the **DNS name** (looks like `flask-demo-alb-xxxxx.eu-west-1.elb.amazonaws.com`)
3. Open `http://<dns-name>/` in your browser

Test from your laptop:

```
ALB_DNS=<your-alb-dns-name>

# Make several requests – you should be hitting different instances
for i in {1..6}; do
  curl http://$ALB_DNS/
  echo
done
```

To **see** the load balancer in action, edit `app.py` locally so the response includes the hostname, push, then

trigger a refresh — but that is Demo 3's job. For now: trust that the ALB is distributing requests.

Step 2.8 — Verify high availability

Let's prove the ASG replaces failed instances automatically:

In the AWS Console:

1. **EC2 -> Instances**
2. Select **one** of the ASG instances (named like `flask-demo-asg`)
3. **Instance state -> Terminate instance -> confirm**

Watch what happens over the next 3-4 minutes:

- The terminated instance disappears
- The ASG notices it has only 1 instance (desired is 2)
- A new instance launches automatically
- The new instance bootstraps, becomes healthy, and joins the Target Group

That is **self-healing infrastructure** — no human intervention needed.

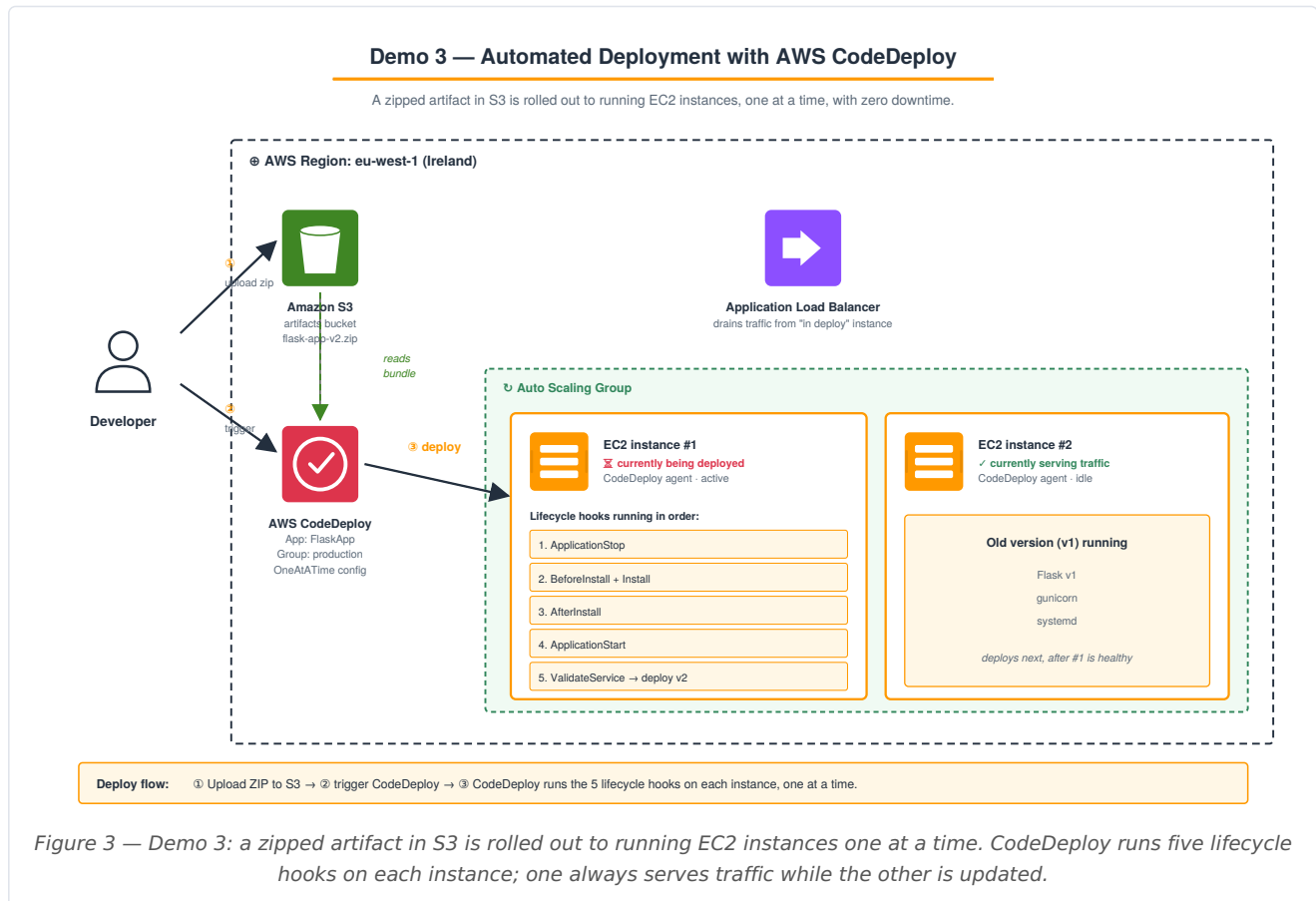
What we built in Demo 2

A load-balanced, self-healing Flask application across multiple AZs. If an instance fails, the ASG replaces it. If traffic doubles, we can change the ASG's desired capacity from 2 to 4 and have twice the capacity in 3 minutes.

Do not delete these resources yet. Demo 3 deploys new versions of the app to this same fleet.

Demo 3 — Automated Deployment with AWS CodeDeploy

Goal: deploy a new version of the Flask app to the ASG with one command — zero downtime, no SSH, no manual restart.



What changes from Demo 2

We extend the project with a deploy specification and lifecycle scripts:

```
flask-app/
├─ app.py
├─ requirements.txt
├─ appspec.yml          # NEW: tells CodeDeploy what to do
├─ deploy/              # NEW: lifecycle scripts
│   ├─ install.sh       # - runs after files are unpacked
│   ├─ start.sh         # - starts the service
│   ├─ stop.sh          # - stops the old version
│   └─ validate.sh      # - checks the new version is healthy
└─ README.md
```

We also need to make a few **infrastructure** changes:

- The EC2 instances need a small agent (the **CodeDeploy agent**) installed
- The instances need an **IAM role** so they can read from S3

- CodeDeploy itself needs an **IAM role** so it can talk to EC2

Step 3.1 — Add the deploy files locally

In your `flask-app` folder on your laptop, create the new files:

File: `appspec.yml`

```
version: 0.0
os: linux

files:
  - source: /
    destination: /home/ubuntu/flask-app

permissions:
  - object: /home/ubuntu/flask-app
    owner: ubuntu
    group: ubuntu

hooks:
  ApplicationStop:
    - location: deploy/stop.sh
      timeout: 60
      runas: root
  AfterInstall:
    - location: deploy/install.sh
      timeout: 300
      runas: root
  ApplicationStart:
    - location: deploy/start.sh
      timeout: 60
      runas: root
  ValidateService:
    - location: deploy/validate.sh
      timeout: 60
      runas: root
```

What is `appspec.yml`? It is CodeDeploy's recipe. It says "unpack the files to this folder, then run these scripts in this order". Without it, CodeDeploy does not know what to do with our zip.

Create the `deploy/` folder and the scripts:

```
mkdir -p deploy
```

File: `deploy/stop.sh`

```
#!/bin/bash
# Stop the running Flask service if it exists
if systemctl is-active --quiet flaskapp; then
    systemctl stop flaskapp
fi
exit 0
```

File: `deploy/install.sh`

```
#!/bin/bash
set -e

# Make sure python3-venv is present (CodeDeploy may run on a fresh AMI)
apt update -y
apt install -y python3 python3-venv python3-pip

cd /home/ubuntu/flask-app

# (Re)create the venv and install dependencies
python3 -m venv venv
./venv/bin/pip install -r requirements.txt

chown -R ubuntu:ubuntu /home/ubuntu/flask-app

# Make sure the systemd service file exists
cat > /etc/systemd/system/flaskapp.service <<EOF
[Unit]
Description=Flask App
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/flask-app
ExecStart=/home/ubuntu/flask-app/venv/bin/gunicorn \\\
    --workers 3 --bind 0.0.0.0:80 app:app
Restart=always

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
```

File: `deploy/start.sh`

```
#!/bin/bash
systemctl enable flaskapp
systemctl start flaskapp
```

File: `deploy/validate.sh`

```
#!/bin/bash
# Wait up to 30 seconds for the app to respond on /health
for i in {1..30}; do
    if curl -fs http://localhost/health > /dev/null; then
        echo "Service is healthy"
        exit 0
    fi
    sleep 1
done

echo "Service did not become healthy in time"
exit 1
```

Make all scripts executable and push to GitHub:

```
chmod +x deploy/*.sh

git add appspec.yml deploy/
git commit -m "Add CodeDeploy lifecycle scripts"
git push origin main
```

Step 3.2 — Modify the app to show the version

So we can [see](#) the deployment happening, update `app.py` to return a version number:

File: `app.py` (updated)

```
import socket
from flask import Flask, jsonify, request

app = Flask(__name__)

APP_VERSION = "v2" # We bump this when we deploy a new version

@app.route("/")
def index():
    return jsonify({
        "message": "Hello from Flask on AWS!",
        "version": APP_VERSION,
        "served_by": socket.gethostname()
    })

@app.route("/echo", methods=["POST"])
def echo():
    data = request.get_json(silent=True) or {}
    return jsonify({"you_sent": data, "version": APP_VERSION})

@app.route("/health")
def health():
    return "ok", 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

Test locally one more time:

```
source venv/bin/activate
python app.py
# In another terminal:
curl http://localhost:8000/
# Expect: {"message": "...", "served_by": "...", "version": "v2"}
```

Stop with `Ctrl+C`. Commit but do not push yet — we will deploy this through CodeDeploy.

```
git add app.py
git commit -m "Add version and hostname to / response"
```

Step 3.3 — Create an S3 bucket for deployment artifacts

CodeDeploy reads our application bundle from S3.

In the AWS Console:

1. Search for **S3** in the top search bar and open it
2. **Create bucket**
3. Bucket name: `flask-demo-artifacts-<your-initials>-<random-number>` (must be globally unique — e.g. `flask-demo-artifacts-cnm-9472`)
4. AWS Region: **eu-west-1**
5. Object Ownership: **ACLs disabled** (default)
6. Block all public access: **leave ticked**
7. Bucket Versioning: **Enable** (lets us roll back to old versions)
8. Tags: `Project = python-chapter-demo`
9. **Create bucket**

Remember this bucket name — we will use it in the CodeDeploy step.

Step 3.4 — Create the IAM role for EC2 instances

The instances need permission to read from our S3 bucket.

In the AWS Console:

1. Search for **IAM** -> open it
2. Left sidebar -> **Roles** -> **Create role**
3. Trusted entity type: **AWS service**
4. Use case: **EC2** -> **Next**
5. **Permissions:** in the search box type and select these two policies (tick each one):
 - `AmazonSSMManagedInstanceCore`
 - `AmazonS3ReadOnlyAccess`
6. **Next**
7. Role name: `EC2CodeDeployRole`
8. Tags: `Project = python-chapter-demo`
9. **Create role**

Why these two policies? `AmazonSSMManagedInstanceCore` lets the CodeDeploy agent talk to AWS.
`AmazonS3ReadOnlyAccess` lets the instance download our zip from S3.

Now attach this role to the Launch Template (so all new instances get it):

1. **EC2 -> Launch Templates -> select** `flask-demo-lt` **-> Actions -> Modify template (Create new version)**
2. Scroll to **Advanced details -> IAM instance profile -> select** `EC2CodeDeployRole`
3. Scroll to **User data** and **add these lines at the END of the existing script:**

```
# Install the CodeDeploy agent (Ubuntu)
apt install -y ruby-full wget
cd /tmp
wget https://aws-codedeploy-eu-west-1.s3.eu-west-1.amazonaws.com/latest/install
chmod +x ./install
./install auto > /tmp/codedeploy-install.log 2>&1
systemctl enable codedeploy-agent
systemctl start codedeploy-agent
```

4. **Create template version**

Now make this new version the default:

1. Still on `flask-demo-lt`, **Actions -> Set default version -> pick the new version number -> Set as default**

Step 3.5 — Refresh the ASG instances so they get the agent

The existing instances do not have the CodeDeploy agent or the IAM role. We replace them.

1. **EC2 -> Auto Scaling Groups -> select** `flask-demo-asg`
2. **Instance refresh tab -> Start instance refresh**
3. Minimum healthy percentage: `50%`
4. **Start instance refresh**

This launches new instances (with the agent + IAM role), waits for each to be healthy, then terminates the old ones. Takes about 5-7 minutes.

Why instance refresh? It replaces instances gradually so the app stays up the whole time.

Step 3.6 — Create the CodeDeploy IAM role

CodeDeploy itself needs permission to talk to EC2.

In the AWS Console:

1. **IAM -> Roles -> Create role**
2. Trusted entity: **AWS service**
3. Use case: from the dropdown select **CodeDeploy**, choose **CodeDeploy** (the basic one, not for Lambda/ECS) -> **Next**
4. **Permissions:** the policy **AWSCodeDeployRole** is auto-attached. Click **Next**.
5. Role name: **CodeDeployServiceRole**
6. Tags: **Project = python-chapter-demo**
7. **Create role**

Step 3.7 — Create the CodeDeploy application and deployment group

In the AWS Console:

1. Search for **CodeDeploy** -> open it
2. Left sidebar -> **Applications -> Create application**
3. Application name: **flask-demo-app**
4. Compute platform: **EC2/On-premises**
5. Tags: **Project = python-chapter-demo**
6. **Create application**

Now create a Deployment Group inside the application:

1. On the application page, scroll to **Deployment groups -> Create deployment group**
2. Deployment group name: **flask-demo-dg**
3. **Service role:** **CodeDeployServiceRole**
4. **Deployment type:** **In-place**
5. **Environment configuration:** tick **Amazon EC2 Auto Scaling groups** -> select **flask-demo-asg**
6. **Deployment configuration:** **CodeDeployDefault.OneAtATime**
7. **Load balancer:** tick **Enable load balancing** -> **Application Load Balancer or Network Load Balancer** -> select **flask-demo-tg**
8. Tags: **Project = python-chapter-demo**
9. **Create deployment group**

What is OneAtATime? CodeDeploy will deploy to one instance at a time. While that instance is updating, the ALB drains its traffic and sends 100% to the others. Once it is healthy, CodeDeploy moves to the next one. **Zero downtime.**

Step 3.8 — Package and upload the app bundle

Back on your laptop, in the **flask-app** folder:

```
# Make sure we have the latest local changes committed (not pushed)
git status # Should show "nothing to commit, working tree clean"

# Create the zip
zip -r flask-app-v2.zip \
    app.py requirements.txt appspec.yml deploy/

# Verify the contents
unzip -l flask-app-v2.zip
```

Upload to S3:

In the AWS Console:

1. **S3** -> open your `flask-demo-artifacts-...` bucket
2. **Upload** -> **Add files** -> select `flask-app-v2.zip`
3. **Upload**

Step 3.9 — Trigger the deployment

In the AWS Console:

1. **CodeDeploy** -> **Applications** -> click `flask-demo-app`
2. Scroll to **Deployment groups**, select `flask-demo-dg`
3. Top right: **Create deployment**
4. Deployment group: `flask-demo-dg` (pre-filled)
5. Revision type: **My application is stored in Amazon S3**
6. Revision location: `s3://<your-bucket-name>/flask-app-v2.zip`
7. Revision file type: `.zip`
8. Leave everything else default
9. **Create deployment**

You will land on a deployment progress page. Watch it for 3-5 minutes:

- Phase 1: deploying to instance #1 (the other one keeps serving traffic)
- Phase 2: deploying to instance #2

Meanwhile, in another tab, open `http://<alb-dns-name>/` and refresh repeatedly. You will see:

- Some requests still return version `v1` from `app.py` (the old version)
- Some requests start returning version `v2` (after deployment to instance #1)
- Eventually all requests return `v2` (after deployment to instance #2)

Zero downtime — the app stayed up the whole time.

What we built in Demo 3

Repeatable, automated, zero-downtime deployment. To deploy a new version:

1. Edit code locally, commit, test

2. `zip -r flask-app-vN.zip ...`
3. Upload to S3
4. Trigger CodeDeploy

That is the deploy pipeline. **Section A** (optional) shows how to put this whole thing on autopilot using GitHub Actions.

Cleanup — Delete everything we created

Run these in order to avoid AWS bills.

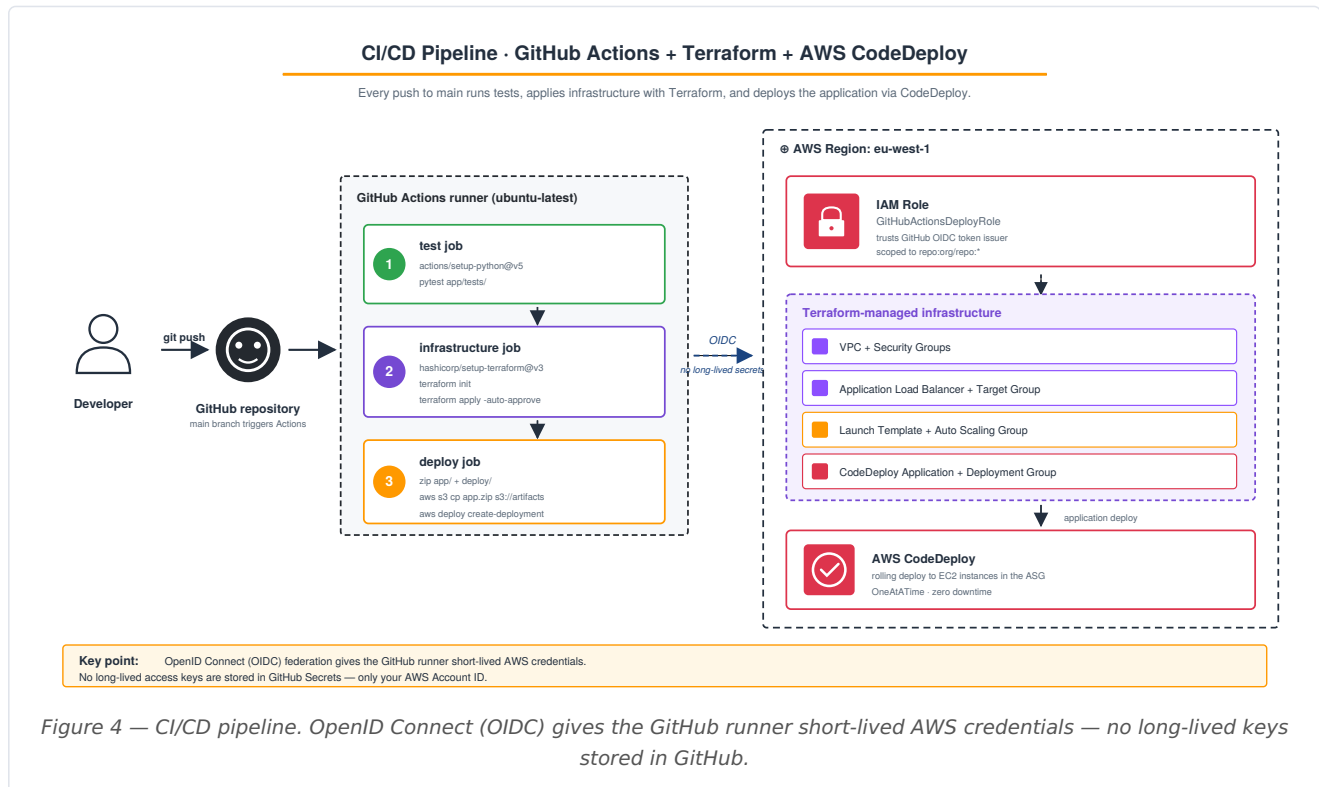
In the AWS Console:

1. **CodeDeploy** -> **Applications** -> select `flask-demo-app` -> **Delete application**
2. **EC2** -> **Auto Scaling Groups** -> select `flask-demo-asg` -> **Delete** -> type "delete" to confirm
3. **EC2** -> **Launch Templates** -> select `flask-demo-lt` -> **Actions** -> **Delete template**
4. **EC2** -> **Load Balancers** -> select `flask-demo-alb` -> **Actions** -> **Delete**
5. **EC2** -> **Target Groups** -> select `flask-demo-tg` -> **Actions** -> **Delete**
6. **EC2** -> **Instances** -> if any remain, select -> **Instance state** -> **Terminate**
7. **EC2** -> **Security Groups** -> delete `flask-alb-sg` and `flask-demo-sg`
8. **EC2** -> **Key Pairs** -> delete `flask-demo-key`
9. **S3** -> open `flask-demo-artifacts-...` -> **Empty** -> then **Delete**
10. **IAM** -> **Roles** -> delete `EC2CodeDeployRole` and `CodeDeployServiceRole`

Tip for next time: use **Tag-based search** to find every resource tagged `Project = python-chapter-demo` and delete in bulk via the **Resource Groups & Tag Editor** service.

Section A — CI/CD with GitHub Actions (optional, ~20 minutes)

Goal: every push to `main` automatically builds, tests, packages, uploads to S3, and triggers a CodeDeploy deployment.



What we add

```
flask-app/  
├── ... (existing files)  
└── .github/  
    ├── workflows/  
    │   └── deploy.yml    # NEW: the CI/CD workflow
```

And we set up an **IAM Role** that GitHub Actions can assume via **OIDC** (OpenID Connect).

Step A.1 — Create the GitHub Actions IAM role

In the AWS Console:

1. **IAM -> Identity providers -> Add provider**
2. Provider type: **OpenID Connect**
3. Provider URL: `https://token.actions.githubusercontent.com`
4. Audience: `sts.amazonaws.com`
5. **Add provider**

Now create the role:

1. **IAM -> Roles -> Create role**
2. Trusted entity: **Web identity**
3. Identity provider: `token.actions.githubusercontent.com`
4. Audience: `sts.amazonaws.com`
5. GitHub organization: `<your-username>`
6. GitHub repository: `flask-app`
7. Branch: `main`
8. **Next**
9. Permissions: attach
 - `AmazonS3FullAccess` (scope this down in production)
 - `AWSCodeDeployDeployerAccess`
10. Role name: `GitHubActionsDeployRole`
11. **Create role**

Copy the role's **ARN** (looks like `arn:aws:iam::123456789012:role/GitHubActionsDeployRole`) — you need it in the next step.

Step A.2 — Add the workflow file

Create the directory and workflow file in your `flask-app` project:

```
mkdir -p .github/workflows
```

File: `.github/workflows/deploy.yml`

```
name: Deploy to AWS

on:
  push:
    branches: [main]

permissions:
  id-token: write # Required for OIDC
  contents: read

env:
  AWS_REGION: eu-west-1
  S3_BUCKET: flask-demo-artifacts-<your-bucket-suffix>
  CODEDEPLOY_APP: flask-demo-app
  CODEDEPLOY_GROUP: flask-demo-dg

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies (for tests)
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest

      - name: Run tests
        run: |
          # Add real tests here. For demo we just import-check.
          python -c "import app; print('App imports OK')"

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<your-account-id>:role/GitHubActionsDeployRole
          aws-region: ${ env.AWS_REGION }

      - name: Package application
        run: |
          zip -r app-${ github.sha }.zip \
            app.py requirements.txt appspec.yml deploy/

      - name: Upload bundle to S3
        run: |
          aws s3 cp app-${ github.sha }.zip \
            s3://${ env.S3_BUCKET }/app-${ github.sha }.zip

      - name: Trigger CodeDeploy deployment
        run: |
          aws deploy create-deployment \
            --application-name ${ env.CODEDEPLOY_APP } \
            --deployment-group-name ${ env.CODEDEPLOY_GROUP } \
            --s3-location bucket=${ env.S3_BUCKET },key=app-${ github.sha }.zip,bundleType=zip
```

Replace `<your-bucket-suffix>` with your actual S3 bucket suffix, and `<your-account-id>` with your 12-digit AWS account ID (find it in the top-right of the Console).

Step A.3 — Push and watch it deploy

```
git add .github/  
git commit -m "Add CI/CD workflow"  
git push origin main
```

On GitHub, open your repository -> **Actions** tab. You should see the workflow running. After 3-5 minutes the deployment finishes and your ALB serves the new version.

From now on, **every** `git push` **to main is a deployment**. No manual zip, no Console clicks.